COMPUTER VISION - LAB 3

Computer Vision 2024-25
P. Zanuttigh, F. Barbato, M. Caligiuri

Topics: Segmentation, Feature Descriptors and Matching.
Goal: Solve the puzzle by placing all the pieces in the proper position.

Write a Python script which includes methods that:

1. Load the puzzle pieces and the reference image from one of the provided datasets.
    a) You can use one of the provided sets: the "*peppa*" one is simpler (except one patch that is a bit more difficult), while "*city_blue*" is slightly more challenging, and "*city_black*" is the most challenging, no need to completely solve all 3
    b) Each set contains a reference image and collections of patches
    c) Notice that the patches can be translated/rotated/scaled
2. Segments from the puzzle pieces' images all the various pieces of the puzzle
3. Extract features from the reference image (you can use SIFT or any other feature descriptor)
4. Extract features from each puzzle piece (you can use SIFT or any other feature descriptor)
5. For each puzzle piece:
    a. Compute the match between the reference image features and the puzzle piece features extracted in (3) and (4). For this, OpenCV offers you the `cv::BFMatcher` class. Remember to use L2 distance for SIFT (the Hamming distance is good for ORB).
    b. Refine the matches found above by selecting the matches with distance less than *ratio * min_distance*, where *ratio* is a user-defined threshold and *min_distance* is the minimum distance found among the matches.
6. You can assume the images and pieces are linked together by an affine transform, using the refined matches, find the transformation between the images. To this end, you can use the RANSAC algorithm. The set of inliers can be computed by using the `findHomography()` function, with `CV_RANSAC` as the third parameter (hint: the inliers can be retrieved by using the mask argument).
7. Using the found homographies, place each piece in the correct position and overlay them to produce an image of the complete puzzle that should resemble the reference one.
8. This is the baseline assignment, see the second page for additional suggestions for extra features allowing to improve your mark.

Optional steps

1. Manually implement RANSAC and affine transform estimation
2. Try various algorithms/strategies to segment the patches, e.g., segmentation, thresholding, morphological operators, object detectors…
3. Try different feature descriptors (e.g., ORB) or try to match in other ways, e.g. using contours
4. Use some blending/mixing technique to improve final image quality
5. Use other techniques for the matching, e.g., template matching or machine learning based approaches
6. Test other images
7. *(very challenging!)* Try to solve without using the reference image
8. Any additional idea is welcome!